

Shiraz University

Department of Computer Science & Engineering

Text Mining & Natural Language Processing

Assignment 2

Implementing Inverted Indexing, TF–IDF, and WordNet-Based Retrieval

Instructor: Dr. Mostafa Fakhrahmad

Teaching Assistants: Alireza Shayan, Hossein Karami

Course: Text Mining & NLP

Term: Spring 1405 / 2026

Start: **Sunday, June 28, 2026 / 7 Tir 1405**

Deadline: **23:59 of Saturday,July 18, 2026 / 27 Tir 1405**

Contents

1 Overview & Learning Goals **3**
- 1.1 Core Objective . . . 3
- 1.2 Learning Goals . . . 3

2 Provided Data **3**
- 2.1 File Structure . . . 3
- 2.2 Data Usage Policy . . . 4
- 2.3 Released Dataset . . . 4
- 2.4 Collection Format . . . 4

3 Scope & Allowed Methods **5**
- 3.1 Required Retrieval Setting . . . 5
- 3.2 Permitted Libraries and Tools . . . 5
- 3.3 AI Tool Policy . . . 6

4 Required System Components **6**
- 4.1 Text Preparation . . . 6
- 4.2 Inverted Index . . . 6
- 4.3 Lexical Retrieval . . . 7
- 4.4 Semantic Retrieval . . . 7
- 4.5 Evaluation . . . 8

5 Bonus: Introduction to Learning to Rank (LTR) **8**
- 5.1 Objective . . . 8
- 5.2 What is Learning to Rank? . . . 9
- 5.3 Task: Implement a Simple LTR Model . . . 9

6 Standard CLI Interface & Code Structure **10**
- 6.1 Command Line Interface . . . 10
- 6.2 Recommended Code Structure . . . 10

7 Required Output Files **11**
- 7.1 `dev_rankings.tsv` . . . 11
- 7.2 `test_rankings.tsv` . . . 12
- 7.3 `dev_metrics.json` . . . 12
- 7.4 `summary.json` . . . 13
- 7.5 `query_trace.jsonl` . . . 13
- 7.6 `report.pdf` . . . 13
- 7.7 `results/extra/` . . . 13

8 query_trace.jsonl — Full Specification **13**
- 8.1 Purpose . . . 13
- 8.2 Top-Level Schema . . . 14
- 8.3 Candidate Document Schema . . . 14
- 8.4 Query Expansion / Semantic Step Policy . . . 15

9 Report Requirements **15**

10 Submission **16**

11 Key Hints **16**

12 Suggested References **16**

1. Overview & Learning Goals

1.1. Core Objective

In this assignment, you will build an **ad hoc information retrieval system** that ranks documents for user queries.

The system must make three kinds of decisions:

- **How to represent text:** Choose a preprocessing and indexing strategy for both documents and queries.
- **How to retrieve:** Rank documents using classical lexical evidence.
- **How to use semantics:** Add a semantic component carefully, without harming ranking quality.

The emphasis is on **retrieval quality**, **interpretability**, and **reproducibility**. A system that adds semantic information blindly may hurt precision. A system that uses semantic evidence selectively and explains its behaviour is rewarded.

1.2. Learning Goals

By completing this assignment, students will:

1. Build and analyse a classical information retrieval pipeline from raw text.
2. Implement an inverted index for efficient term-to-document access.
3. Implement TF–IDF ranking with cosine similarity.
4. Use WordNet-based semantic information for retrieval improvement.
5. Evaluate ranked results using standard IR metrics.
6. Perform ablation studies and error analysis.
7. Produce fully reproducible, well-documented outputs.

2. Provided Data

2.1. File Structure

```
HW2_data/
|-- documents.jsonl     <- document collection
|-- queries_dev.tsv     <- development queries
|-- qrels_dev.tsv       <- relevance judgments for dev queries
|-- queries_test.tsv    <- test queries
+-- README_dataset.txt <- dataset description and field
   definitions
```

Listing 1 – *Dataset directory layout*

All files are UTF-8 encoded.

2.2. Data Usage Policy

- `documents.jsonl` is the only document collection that may be indexed and searched.
- `queries_dev.tsv` and `qrels_dev.tsv` are for model design, parameter tuning, and error analysis.
- `queries_test.tsv` may be used for producing final ranked outputs.
- Students must use the released package as provided.
- Query-specific hand-crafted rules are not allowed.

2.3. Released Dataset

This assignment is distributed with a course-prepared dataset derived from **BEIR / SCIDOCS**.

The released student package contains:

- `documents.jsonl`
- `queries_dev.tsv`
- `qrels_dev.tsv`
- `queries_test.tsv`
- `README_dataset.txt`

> ▹ **Note**
>
> Document representation in this homework should use `title + text`. A small number of documents may have an empty `text` field while still having a valid `title`; such records are part of the released dataset and should still be indexed using the available title field.

2.4. Collection Format

Each document record contains:

- `doc_id`
- `title`
- `text`

Each query record contains:

- `query_id`
- `query_text`

Each qrels row contains:

- `query_id`
- `doc_id`
- `relevance`

Relevance is binary in this assignment: `1 = relevant, 0 = non-relevant or unjudged`.

3. Scope & Allowed Methods

3.1. Required Retrieval Setting

Your final system must contain:

- a lexical retrieval component based on an inverted index,
- TF–IDF ranking with cosine similarity as the main required ranking method,
- a semantic retrieval component using WordNet.

The final submitted system must produce two ranked runs on the dev set:

1. a lexical baseline run,
2. a final semantic-aware run.

The semantic component must improve or meaningfully analyse retrieval behaviour. Blind semantic expansion without analysis is not sufficient.

3.2. Permitted Libraries and Tools

The following are allowed:

Table 1 – *Allowed tools and libraries*

Role	Examples
Tokenization / stemming / stopword handling	NLTK, regular expressions, spaCy (tokenization only)
Numerical computation	NumPy, SciPy
Verification only	scikit-learn for checking TF–IDF or cosine similarity
WordNet access	NLTK WordNet
Plotting / tables	matplotlib, seaborn, pandas

The following must be implemented by you:

- inverted index,
- TF–IDF weighting and cosine ranking,
- core IR evaluation metrics,
- at least one semantic retrieval or semantic reranking strategy based on WordNet.

3.3. AI Tool Policy

AI tools (chatbots, coding assistants, etc.) are **allowed without restriction**. However:

- Students must be able to **fully explain** any submitted code or design choice.
- AI usage must be **disclosed** in the report.
- Inability to explain submitted work during oral evaluation may result in grade penalties.

4. Required System Components

4.1. Text Preparation

Implement a configurable preprocessing pipeline for both documents and queries.

Required steps:

1. Tokenization using the regex pattern `(?u)\b\w\w+\b`
2. Case folding
3. Stop-word removal
4. Porter stemming

Requirements:

- Each step must be individually enabled or disabled.
- The same pipeline must be applied consistently to documents and queries.
- Report vocabulary size before and after preprocessing.
- Show at least one example query and one example document before and after preprocessing.

4.2. Inverted Index

Build an inverted index from scratch.

Minimum requirements:

- For each term, store the posting list of document IDs.
- Store term frequency for each term–document pair.
- Store document frequency for each term.
- Report:
  - number of unique terms,
  - number of postings,
  - 3 sample posting lists,
  - top 10 terms by document frequency.

The index must be used by your lexical retrieval component.

4.3. Lexical Retrieval

Implement TF–IDF ranking with cosine similarity from scratch.

Requirements:

- Compute TF–IDF vectors for documents.
- Compute TF–IDF vectors for queries using the same vocabulary and IDF values.
- Rank all documents for each query by cosine similarity.
- Produce the top 100 ranked documents for each dev query.
- Display the top 10 results for 3 sample dev queries, along with relevance labels.

★ Important Note

Important: You must use your inverted index to efficiently retrieve candidate documents. Computing cosine similarity iteratively over the entire unindexed document collection (linear scan) is strictly prohibited.

You must also analyse the effect of preprocessing choices on lexical retrieval quality.

4.4. Semantic Retrieval

Implement one semantic component using WordNet.

Choose one of the following two options:

Option A — Query Expansion

Use WordNet synsets or lemmas to expand selected query terms.

Requirements:

- Define a policy for which query terms may be expanded.
- Limit expansion aggressively; expanding every term is not recommended.
- Explain how you score original terms versus expansion terms.
- Compare retrieval quality before and after expansion.

Option B — Semantic Reranking

Use WordNet-based semantic similarity to rerank the top lexical candidates retrieved from the initial classical IR phase. To ensure computational feasibility, you must restrict the semantic alignment process to content words and a limited candidate pool.

Mathematical Formulation:

1. **Candidate Pool Selection:** For a given query Q, retrieve the top $K = 20$ documents using your implemented TF–IDF cosine similarity ranker. Let this set of documents be $\mathcal{D}_{\text{lexical}}$.
2. **Content Word Extraction:** Extract filtered sets of unique terms from the query (Q_{words}) and the document (D_{words}). To avoid combinatorial explosion, you must only consider tokens tagged as **Nouns** or **Verbs** after case-folding and stemming:
$$Q_{\text{words}} = \{q \in Q \mid \text{POS}(q) \in \{\text{Noun}, \text{Verb}\}\} \tag{1}$$
3. **Asymmetric Semantic Score:** Compute the semantic relatedness between Q and a candidate document $D \in \mathcal{D}_{\text{lexical}}$ by finding the maximum Wu–Palmer

similarity (Sim_{WP}) alignment for each query term:

$$\text{Score}_{\text{semantic}}(Q, D) = \frac{1}{|Q_{\text{words}}|} \sum_{q \in Q_{\text{words}}} \max_{d \in D_{\text{words}}} \text{Sim}_{\text{WP}}(q, d) \tag{2}$$

where $\text{Sim}_{\text{WP}}(q, d)$ represents the highest similarity score obtained across all synset pairs of the word senses for q and d in WordNet.

4. **Linear Combination Reranking:** Calculate the final integrated retrieval score using a weighted combination controlled by the hyperparameter $\alpha \in [0, 1]$:

$$\text{Score}_{\text{final}}(Q, D) = \alpha \cdot \text{Score}_{\text{lexical}}(Q, D) + (1 - \alpha) \cdot \text{Score}_{\text{semantic}}(Q, D) \tag{3}$$

Requirements:

- Rerank only the top 20 documents for each query in the dev set.
- Document how your pipeline handles cases where Q_{words} or D_{words} becomes empty after POS filtering.
- Perform a grid sweep over α (e.g., from 0.0 to 1.0 with a step of 0.2) and report its impact on MAP and NDCG@10.
- Identify and discuss at least two queries where this semantic formulation successfully fixed lexical mismatches, and two queries where it introduces noise or polysemy errors.

4.5. Evaluation

Evaluate your retrieval systems using standard IR metrics.

Requirements:

- Implement Mean Average Precision (MAP), Mean Reciprocal Rank (MRR), Precision@k (for $k = 1, 5, 10$), and NDCG@k (for $k = 5, 10$) from scratch.
- Compute and report the average of these metrics across all queries in the dev set.

> ▹ **Note**
>
> Note on NDCG: Since our relevance judgments are binary, the relevance score (gain) for a document is exactly its binary label (1 if relevant, 0 otherwise).

5. Bonus: Introduction to Learning to Rank (LTR)

5.1. Objective

The primary objective of this bonus section is to provide students with a foundational understanding of **Learning to Rank (LTR)** techniques within the context of information retrieval.

5.2. What is Learning to Rank?

Learning to Rank approaches leverage machine learning to automatically learn a ranking model from labeled data. Unlike traditional IR methods that rely on hand-crafted scoring functions (like TF-IDF or semantic similarity), LTR models learn to combine various features of a query-document pair (e.g., term overlap, document length, semantic relatedness, link structure, etc.) to predict a relevance score. These models are typically trained in a supervised manner, using queries with known relevance judgments. Common LTR approaches include pointwise (predicting relevance for each document independently), pairwise (predicting the relative order of two documents), and listwise (optimizing the entire ranked list) methods. The goal is to generalize from labeled examples to unseen queries and documents, ultimately improving retrieval effectiveness.

5.3. Task: Implement a Simple LTR Model

For this bonus, you are encouraged to implement a simplified LTR model. You can choose one of the following:

1. **Pointwise Regression:** Train a regression model (e.g., Linear Regression, Ridge, or a simple feedforward neural network) to predict the relevance score for a given query-document pair. You will need to extract features such as:
   - TF-IDF scores (e.g., cosine similarity)
   - WordNet-based semantic similarity scores (as computed in Section 4.4)
   - Document length features
   - Query-document term overlap counts

   Train this model on the `queries_dev.tsv` and `qrels_dev.tsv` data.
2. **Pairwise Ranking:** Implement a simple pairwise model that learns to predict whether document A should be ranked higher than document B for a given query. Models like RankSVM or RankBoost can be considered, but simpler logistic regression on pairwise features is also acceptable.

Requirements:

- Use the features derived from your TF-IDF and semantic components.
- Train your model(s) using the provided development data (`queries_dev.tsv`, `qrels_dev.tsv`).
- Generate ranked lists for the dev and test sets using your trained LTR model. Ensure your LTR model is applied to the top-K candidates retrieved by TF-IDF, similar to the reranking approach.
- Report the performance (MAP, NDCG@10) of your LTR model alongside the other baselines in `dev_metrics.json` and `test_rankings.tsv` (using a new method name, e.g., `ltr`).
- Discuss the challenges encountered and the effectiveness of your chosen LTR approach in your report.

Note: The focus of this bonus is on understanding the LTR workflow (feature extraction, training, evaluation). Complex LTR models or extensive hyperparameter tuning are not required. A well-documented implementation of a simple LTR model

that produces reasonable results will be sufficient.

6. Standard CLI Interface & Code Structure

6.1. Command Line Interface

The entire pipeline must be runnable via a single command:

```
python system.py \
  --docs         HW2_data/documents.jsonl \
  --dev-queries  HW2_data/queries_dev.tsv \
  --dev-qrels    HW2_data/qrels_dev.tsv \
  --test-queries HW2_data/queries_test.tsv \
  --outdir       results
```

Listing 2 – *Main CLI invocation*

Minimum required behaviour:

- End-to-end execution without manual intervention.
- All output files written to `--outdir`.
- Reproducible results (fix random seeds if any randomness is used).

Recommended optional arguments:

```
--seed 42
--config config.json
--topk 100
--semantic-mode rerank
--alpha 0.8
--beta 0.2
```

Listing 3 – *Optional CLI arguments*

6.2. Recommended Code Structure

Although the entry point is `system.py`, you are **strongly encouraged** to organise your implementation across multiple modules rather than placing all code in a single file.

```
project_root/
|-- system.py
|-- preprocess.py
|-- index.py
|-- ranker.py
|-- semantic.py
|-- evaluator.py
|-- config.json
+-- results/
|   |-- dev_rankings.tsv
|   |-- test_rankings.tsv
```

```
|   |-- dev_metrics.json
|   |-- summary.json
|   |-- query_trace.jsonl
|   |-- report.pdf
+-- extra/
```

Listing 4 – *Recommended project structure*

▹ **Note**

Module responsibilities:

- `system.py` — CLI parsing and end-to-end pipeline.
- `preprocess.py` — tokenization and preprocessing.
- `index.py` — inverted index construction and lookup.
- `ranker.py` — lexical ranking.
- `semantic.py` — expansion or reranking logic.
- `evaluator.py` — metric computation and output writing.

All modules must be importable without side effects.

7. Required Output Files

All output files must be written to the `--outdir` directory (default: `results/`).

7.1. `dev_rankings.tsv`

One row per ranked `(query_id, doc_id)` pair in the dev results.

Table 2 – *Schema for* `dev_rankings.tsv`

Column	Type	Description
`query_id`	string/int	Query identifier
`doc_id`	string/int	Document identifier
`rank`	int	Rank position (starting from 1)
`score`	float	Retrieval score
`method`	string	`random`, `overlap`, `tfidf`, or `final`

7.2. `test_rankings.tsv`

Same format as `dev_rankings.tsv`, but only for the final system.

7.3. `dev_metrics.json`

A JSON object containing all evaluation metrics for all required systems:

Listing 5 – *Example* `dev_metrics.json`

```
{
  "random": {
    "MAP": 0.031,
    "MRR": 0.072,
    "P@1": 0.020,
    "P@5": 0.040,
    "P@10": 0.053,
    "NDCG@5": 0.060,
    "NDCG@10": 0.079
  },
  "overlap": {
    "MAP": 0.112,
    "MRR": 0.201,
    "P@1": 0.140,
    "P@5": 0.116,
    "P@10": 0.098,
    "NDCG@5": 0.153,
    "NDCG@10": 0.174
  },
  "tfidf": {
    "MAP": 0.201,
    "MRR": 0.342,
    "P@1": 0.280,
    "P@5": 0.224,
    "P@10": 0.187,
    "NDCG@5": 0.266,
    "NDCG@10": 0.291
  },
  "final": {
    "MAP": 0.219,
    "MRR": 0.361,
    "P@1": 0.300,
    "P@5": 0.232,
    "P@10": 0.193,
    "NDCG@5": 0.279,
    "NDCG@10": 0.307
  }
}
```

7.4. `summary.json`

System configuration and metadata:

Listing 6 – *Example* `summary.json`

```
{
  "student_name": "...",
  "student_id": "...",
  "token_pattern": "(?u)\\b\\w\\w+\\b",
  "lowercase": true,
  "stopword_removal": true,
  "stemming": true,
  "lexical_method": "tfidf_cosine",
  "semantic_method": "wordnet_rerank",
  "topk": 100,
  "random_seed": 42,
  "score_weights": {
    "lexical": 0.8,
    "semantic": 0.2
  }
}
```

7.5. `query_trace.jsonl`

Detailed per-query diagnostic log. See Section 8 for full specification.

7.6. `report.pdf`

Written report. See Section 9 for full specification.

7.7. `results/extra/`

Optional directory for additional outputs: plots, ablation logs, intermediate files, and extra analysis.

8. query_trace.jsonl — Full Specification

This is the most important output file for interpretability and debugging. Each line must be a valid JSON object representing one query.

8.1. Purpose

The trace file must answer the following questions for every query:

- How was the query preprocessed?
- Which lexical results were retrieved first?
- Was semantic expansion or reranking applied?
- Which documents changed position after the semantic step?
- Why did the final ranking change?

8.2. Top-Level Schema

Listing 7 – *Top-level* `query_trace.jsonl` *schema*

```
{
  "query_id": 12,
  "query_text": "aircraft stability problems",
  "query_tokens_raw": ["aircraft", "stability", "problems"],
  "query_tokens_final": ["aircraft", "stabil", "problem"],
  "method": "wordnet_rerank",
  "lexical_top10": [
    {"doc_id": 101, "score": 2.81, "rank": 1},
    {"doc_id": 732, "score": 2.45, "rank": 2}
  ],
  "semantic_step": {
    "applied": true,
    "type": "rerank",
    "details": "top 20 lexical documents reranked using WordNet
       similarity"
  },
  "final_top10": [
    {"doc_id": 732, "score": 2.93, "rank": 1},
    {"doc_id": 101, "score": 2.88, "rank": 2}
  ],
  "summary": {
    "num_docs_scored_lexically": 1398,
    "num_docs_semantically_reranked": 20,
    "top10_changed": true
  }
}
```

8.3. Candidate Document Schema

If semantic reranking is used, include a document-level explanation for at least the top 5 final documents:

Listing 8 – *Per-document reranking entry*

```
{
  "doc_id": 732,
  "lexical_score": 2.45,
  "semantic_score": 0.61,
  "combined_score": 2.93,
  "rank_before": 2,
  "rank_after": 1,
  "reason": "contains high-overlap lexical terms and semantically
     related term 'control'"
}
```

8.4. Query Expansion / Semantic Step Policy

- If expansion is used, record the original query terms and all added expansion terms.
- If reranking is used, record the lexical candidate pool size.
- Scores must be JSON numbers, not strings.
- The trace must be consistent with the final submitted ranking files.

9. Report Requirements

The report (`report.pdf`) must be a self-contained document covering the following sections:

1. **Problem Setting:** Explain ad hoc information retrieval and why purely lexical matching is often insufficient.
2. **Method:** Describe the full pipeline from raw query/document text to final ranking.
3. **Preprocessing Design:** Explain tokenization, stop-word removal, stemming, and any alternative configurations.
4. **Indexing and Lexical Retrieval:** Describe the inverted index and TF–IDF ranking method.
5. **Semantic Method:** Explain the WordNet-based expansion or reranking strategy, including design trade-offs.
6. **Experimental Settings:** Report all important hyperparameters and design choices.
7. **Results:** Provide a metric table comparing all required baselines and the final system.
8. **Ablation Study:** Include at least **3 meaningful ablations**. Examples:
   - with vs. without stop-word removal,
   - with vs. without stemming,
   - overlap vs. TF–IDF,
   - TF–IDF vs. TF–IDF + semantic step,
   - aggressive vs. conservative semantic expansion.
9. **Error Analysis:** Include at least **5 concrete dev queries**:
   - at least 2 cases where semantic information helped,
   - at least 2 cases where semantic information hurt,
   - at least 1 other interesting case.

   For each case, show the query, selected top results, relevance pattern, and the root cause.
10. **Reflection:** State one systematic weakness of the system and one concrete improvement.
11. **AI Usage Disclosure:** Describe how AI tools and their specific versions were used (e.g., "used Gemini 3 Pro for boilerplate code," "used ChatGPT-4 to debug the TF–IDF formula," "used Claude 4.6 Sonnet for brainstorming," or similar usage with Copilot, Grok, Kimi, etc.). This section is for transparency, not pe-

nalisation.

▹ **Note**

A system with a simpler but well-justified semantic step, clean reproducibility, and strong analysis is valued more than a complex pipeline with weak explanation.

10. Submission

★ **Important Note**

Submit a single zip archive named:

`NLP-HW2-FirstName_LastName-StudentNumber.zip`

The archive must contain:

- all code files,
- the `results/` directory,
- the PDF report.

Do not include raw dataset files unless explicitly requested.

11. Key Hints

- **Strong preprocessing does not always improve retrieval.** Evaluate, do not assume.
- **Stemming may help recall but may hurt exact lexical precision.**
- **Blind semantic expansion often lowers precision.**
- **Semantic reranking is usually safer than full-corpus semantic scoring.**
- **A simple TF–IDF baseline is hard to beat if semantic signals are noisy.**
- **Read your `query_trace.jsonl` manually before trusting your metric table.**
- **Reproducibility matters.** The graders will rerun your system.

12. Suggested References

1. C. D. Manning, P. Raghavan, and H. Sch"utze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
2. K. Sp"arck Jones and P. Willett (eds.), *Readings in Information Retrieval*. Morgan Kaufmann, 1997. *(for historical context on Cranfield-style evaluation)*
3. P. Bajaj et al., "MS MARCO: A Human Generated MAchine Reading COmprehension Dataset," *arXiv preprint arXiv:1611.09268*, 2016.
4. N. Thakur et al., "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation

of Information Retrieval Models," *SIGIR*, 2021.

5. R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.
6. W. B. Croft, D. Metzler, and T. Strohman, *Search Engines: Information Retrieval in Practice*. Addison-Wesley, 2010.
7. D. Jurafsky and J. H. Martin, *Speech and Language Processing*, chapters on information retrieval and lexical semantics.